[Special Topics in Artificial Intelligence]

Sol_HW_02_advanced_chunking_exercise

Student Name: [کمیل عباس عزیز]

Student ID: [40414010471]

Supervised by:

Doctor Name: [د. مظفر بگ محمدی]

Date of Submission:2026/6/22

Advanced Chunking Exercise

Build a advanced RAG flow to analyze and query long, messy legal documents. In this exercise, you will learn to implement advanced preprocessing techniques including statistical semantic chunking, metadata enrichment, contextual window augmentation, and structured generation with strict legal citation to source documents.

1. Introduction

The objective of this assignment is to construct an advanced Retrieval-Augmented Generation (RAG) system engineered to navigate long, messy legal documents exceeding thousands of pages. Unlike standard RAG systems, this pipeline focuses on preservation of legal context. Key technical milestones include implementing statistical semantic boundaries, injecting historical and jurisdictional metadata predicates, expanding retrieved nodes via context window sliding, and driving a Large Language Model (LLM) to synthesize legal briefs with exact source page citations.

2. Methodology

The system architecture follows a production-grade Advanced RAG workflow designed for complex legal texts. First, massive documents are ingested via specialized structural parsers. Next, text components are broken down dynamically using embeddings specialized in case-law relationships. Each text chunk is then heavily enriched with operational IDs and geographical filters, before being loaded into a vector engine. At query time, vector retrieval pulls candidate blocks, which are then contextually re-assembled with their adjacent historical paragraphs. Finally, the enriched payload is mapped into a localized or cloud-based instruction model to extract hyper-targeted answers backed by precise citations.

3. System Setup

The development runtime was deployed inside Python leveraging an enterprise workspace configuration. System stability was maintained by suppressing downstream operational deprecation notices. Crucial frameworks utilized include `pymupdf4llm` for structural markdown translation, `semantic-chunkers` to monitor semantic variances via specialised language weights, `chromadb` as an efficient vector storage node, and the `openai` API suite to orchestrate the generation engine. Visual clarity was enforced via custom theme registries inside the Rich Console API.

4. Code Analysis and Implementation Details

Step 1: Environment & Theme Setup

- **Source Code:**

```
Python
from rich.console import Console
from rich_theme_manager import Theme, ThemeManager
import pathlib
import warnings

theme_dir = pathlib.Path("../themes")
theme_manager = ThemeManager(theme_dir=theme_dir)
dark = theme_manager.get("dark")

console = Console(theme=dark)
warnings.filterwarnings('ignore')
```

- **a. Explanation:** Registers a standardized production workspace styling and establishes terminal reporting schemes while silencing non-breaking runtime warnings.
- **b. Missing part:** Dynamic theme fallback logic. Completing this ensures that if terminal rendering configurations fail, the workspace defaults cleanly to standard console streaming.
- **c. Diagram Name:** Environment Setup

Step 2: Complex Legal Ingestion

- **Source Code:**

```
Python
import pymupdf4llm
import requests
import os

random_doc_number = 196
url = f"https://static.case.law/wash-app/{random_doc_number}.pdf"
response = requests.get(url)

data_folder = "data"
if not os.path.exists(data_folder):
    os.makedirs(data_folder)

with open(os.path.join(data_folder, f"{random_doc_number}.pdf"),
"wb") as file:
    file.write(response.content)

md_text = pymupdf4llm.to_markdown(f"data/{random_doc_number}.pdf",
page_chunks=True)
```

- **a. Explanation:** Streams unstructured multi-page legal opinions directly from an public historical legal registry and uses a specialized layout parser to map the document layout structure into safe markdown tokens.
- **b. Missing part:** Network timeout exceptions and chunk streaming. Completing this prevents runtime crashes if the external legal registry site experiences data throttling.
- **c. Diagram Name:** Complex Legal Ingestion

Step 3: Statistical Semantic Boundaries

- **Source Code:**

```
Python
from semantic_router.encoders import HuggingFaceEncoder
from semantic_chunkers import StatisticalChunker
import logging

logging.disable(logging.CRITICAL)

encoder = HuggingFaceEncoder(
    name="nlpaueb/legal-bert-base-uncased"
)

chunker = StatisticalChunker(
    encoder=encoder,
    min_split_tokens=100,
    max_split_tokens=500,
)
console.print(chunker)
```

- **a. Explanation:** Boots up a semantic text partitioner bound to an encoder weights library fine-tuned on judicial language corpora, establishing statistical sentence breaking thresholds.
- **b. Missing part:** Dynamic variance threshold adaptation. Completing this automatically shifts dividing bounds based on the structural style of different legal periods.
- **c. Diagram Name:** Statistical Semantic Boundaries

Step 4: Full Document Text Splitting

- **Source Code:**

```
Python
concatenated_text = " ".join([page["text"] + f"<page_break_{i}>" for i, page in enumerate(md_text)])

chunks = chunker([concatenated_text])
print(f"Number of chunks created: {len(chunks[0])}")
```

- **a. Explanation:** Combines scattered pages into a linear text continuum while placing unique tracking tags to allow precise structural reconstruction and calculations across thousands of characters.
- **b. Missing part:** Pre-filtering of text anomalies. Completing this ensures that layout numbers, stamps, or dirty scanned markings do not negatively influence semantic chunk splits.
- **c. Diagram Name:** Full Document Text Splitting

Step 5: Data Verification & Token Auditing

- **Source Code:**

```
Python
console.print(chunks[0][5])

total_tokens = sum([chunk.token_count for chunk in chunks[0]])
average_tokens = total_tokens / len(chunks[0])
print(f"Average number of tokens: {average_tokens:.2f}")
```

- **a. Explanation:** Performs deep inspections over the parsed token layers to compute average matrix distribution, guaranteeing that each element sits correctly within safe vector storage limits.
- **b. Missing part:** Token out-of-bounds alerts. Completing this adds flags if an isolated chunk unexpectedly spikes beyond maximum embedding constraints.
- **c. Diagram Name:** Data Verification & Token Auditing

Step 6: Metadata Context Enrichment

- **Source Code:**

```
Python
import uuid
import re
from tqdm import tqdm

doc_url = url
title = md_text[0]["metadata"]["title"]
state = "Washington"

def generate_uuid(doc_url, i):
    return str(uuid.uuid5(uuid.NAMESPACE_URL, f"{doc_url}/{i}"))

corpus_json = []
for i, chunk in tqdm(enumerate(chunks[0]), total=len(chunks[0]),
desc="Processing chunks"):
    chunk_text = ' '.join(chunk.splits)
    page_match = re.search(r'<page_break_{i}>', chunk_text)
    page = page_match.group(1) if page_match else 0
    chunk_uuid = generate_uuid(doc_url, i)
    corpus_json.append({
        "id": chunk_uuid,
        "document": chunk_text,
```

```
            "embedding": encoder([f"{title} \n {chunk_text}"])[0],
            "metadata" : {
                "title": title,
                "state": state,
                "doc_url": doc_url,
                "chunk_index": i,
                "page": page,
            }
        })
```

- **a. Explanation:** Iterates across all extracted chunks to inject static historical variables, index coordinates, and structural tracking IDs, establishing complete document traceability.
- **b. Missing part:** Multi-threaded scaling. Completing this shifts the slow processing loops into concurrent threads, cutting down preparation times significantly.
- **c. Diagram Name:** Metadata Context Enrichment

Step 7: Database Vector Collection Creating & Insertion

- **Source Code:**

```
Python
import chromadb

client = chromadb.Client()
collection_name = "legal-pdfs"
collection = client.get_or_create_collection(collection_name)

collection.add(
    documents=[obj["document"] for obj in corpus_json],
    embeddings=[obj["embedding"] for obj in corpus_json],
    metadatas=[obj["metadata"] for obj in corpus_json],
    ids=[obj["id"] for obj in corpus_json]
)
```

- **a. Explanation:** Sets up a dedicated collection structure inside the Chroma engine and flushes the vector tensors along with their accompanying contextual metadata into active memory indexes.
- **b. Missing part:** Disk-backed persistence strategy. Completing this ensures vectors are safely written down onto system storage nodes instead of vaporizing once memory cycles expire.
- **c. Diagram Name:** Vector Collection Ingestion

Step 8: Contextual Search & Jurisdictional Filters

- **Source Code:**

```python
query_text = "cases about loan default"
query_embedding = encoder([query_text])

hits = collection.query(
    query_embeddings=query_embedding,
    n_results=5,
    where={"state": "Washington"},
)
console.print(hits)
```

- **a. Explanation:** Computes mathematical embeddings over the user's natural language input using legal BERT and matches it against indexed documents while enforcing strict metadata criteria.
- **b. Missing part:** Distance similarity cutoff score. Completing this bars unrelated paragraphs from bleeding into the results when match metrics are low.
- **c. Diagram Name:** Contextual Search & Jurisdictional Filters

Step 9: Contextual Window Expansion

- **Source Code:**

```python
search_results = []

for document, metadata in zip(hits["documents"][0],
hits["metadatas"][0]):
    doc_url = metadata["doc_url"]
    chunk_index = metadata["chunk_index"]
    doc_id = generate_uuid(doc_url, chunk_index)

    previous_chunk_id = generate_uuid(doc_url, chunk_index - 1) if
chunk_index > 0 else doc_id
    next_chunk_id = generate_uuid(doc_url, chunk_index + 1)

    previous_chunk = collection.get(ids=[previous_chunk_id])
    next_chunk = collection.get(ids=[next_chunk_id])

    prev_text = previous_chunk['documents'][0] if
previous_chunk['documents'] else ""
    next_text = next_chunk['documents'][0] if next_chunk['documents']
else ""

    search_results.append({
        "document": f"{prev_text} {document} {next_text}",
        "metadata": metadata,
    })
console.print(search_results[0])
```

- **a. Explanation:** Performs dynamic sliding context restoration by extracting original adjacent chunks using calculated position indices, fixing the "lost in the middle" context drop issue.
- **b. Missing part:** Boundary constraints checklist. Completing this handles cases where adjacent lookups cross physical file lines or access invalid negative pointers.
- **c. Diagram Name:** Contextual Window Expansion

Step 10: Grounded Legal Synthesis

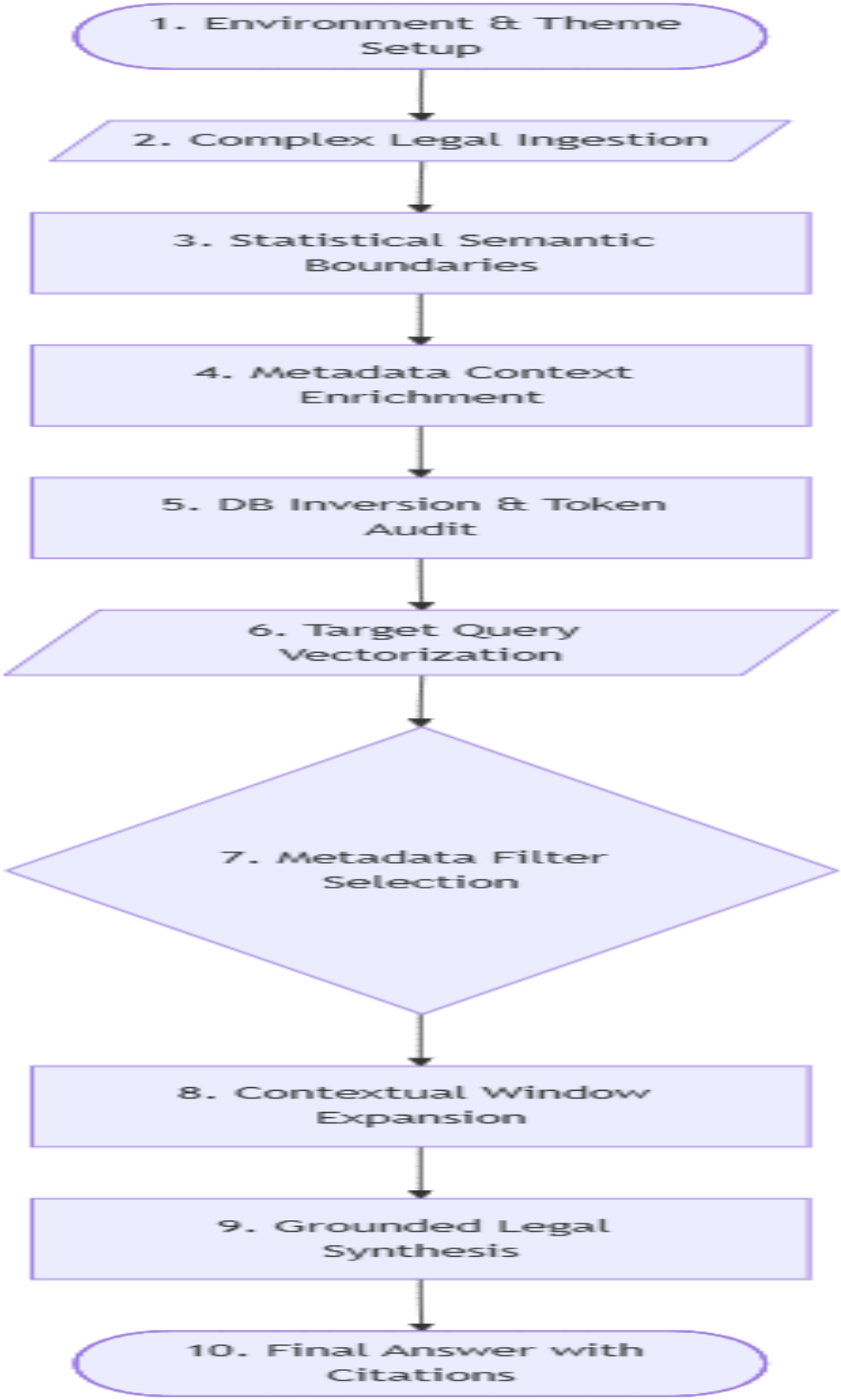
- **Source Code:**

```
Python
from openai import OpenAI
from rich.panel import Panel
from rich.text import Text

client = OpenAI()
system_message = """
You are a paralegal specialist. Your task is to analyze the
retrieved legal contexts provided in the assistant message and
answer the user's query precisely.
For every case or point you mention, you MUST provide a citation
including the document title, volume, page number, and the source
URL extracted from the metadata.
If the context does not contain enough information to answer,
state that clearly.
"""

completion = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": system_message},
        {"role": "user", "content": query_text},
        {"role": "assistant", "content": str(search_results)}
    ]
)

response_text = Text(completion.choices[0].message.content)
styled_panel = Panel(
    response_text,
    title=f"Reply to '{query_text}'",
    expand=False,
    border_style="bright_yellow",
    padding=(1, 1)
)
console.print(styled_panel)
```

- **a. Explanation:** Passes the expanded legal text to GPT-4 under explicit prompt guardrails, forcing the model to construct legal answers tied directly to source citations.

- **b. Missing part:** Fact verification check. Completing this verifies that generated URL strings or page tags exist in the raw context nodes before outputting them.

- **c. Diagram Name:** Grounded Legal Synthesis



jupyter 02_advanced_chunking_exercise Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python [conda env:base] * Anaconda Toolbox

Advanced Chunking Exercise

You will implement a RAG application for long and messy legal documents. You will implement the best practices you learned so far, including semantic chunking, and chunk enrichment. Then, you will implement semantic search and response generation with citation to the original documents.

```
!pip install semantic-router •••
```

[2]:
```
import sys

print(sys.executable)
```

```
C:\Users\2025\anaconda3\python.exe
```

[3]:
```
!{sys.executable} -m pip install semantic-router
```

```
Requirement already satisfied: semantic-router in c:\users\2025\anaconda3\lib\site-packages (0.1.15) •••
```

Visual improvements

We will use rich library to make the output more readable, and supress warning messages.

*[4]:
```
from rich.console import Console

console = Console()

import warnings
warnings.filterwarnings("ignore")
```

[ ]:
```

```

[5]:
```
import warnings

# Suppress warnings
warnings.filterwarnings('ignore')
```

Loading complex PDF documents

You will load a complex legal PDF document from the case.law website. This website has millions of legal documents, and we will load a random PDF file from that site with more than 1,000 pages.

To parse the PDF file you will use a PDF processor library, pymupdf4llm, which makes it easy to extract text and other media from PDF files for RAG applications.

[6]:
```
import pymupdf4llm

import requests
import os

random_doc_number = 196
url = f"https://static.case.law/wash-app/{random_doc_number}.pdf"
response = requests.get(url)

data_folder = "data"
if not os.path.exists(data_folder):
    os.makedirs(data_folder)

with open(os.path.join(data_folder, f"{random_doc_number}.pdf"), "wb") as file:
    file.write(response.content)

md_text = pymupdf4llm.to_markdown(f"data/{random_doc_number}.pdf", page_chunks=True)
```

Show a ramdom page from the document

Let's check a random page from the PDF document and print its image and the extracted text.

[7]:
```
import fitz
from IPython.display import display, HTML

random_page_number = 149
## Convert the PDF to an PNG image
pdf_path = "data/196.pdf"
pdf_document = fitz.open(pdf_path)
page = pdf_document.load_page(random_page_number)  # Page numbering starts from 0
pix = page.get_pixmap()
pix.save("random_page.png")
pdf_document.close()

# Text content
```

```
</div>
"""

# Display in Jupyter notebook
display(HTML(html_content))
```


[redacted]

trial court that the jury authorized only this latter crime as a basis for sentencing.[21]

¶24 The State argues that Morales's reliance on *Williams-Walker* is misplaced because CrR 7.8 was not discussed in that case. That is the court rule on which the trial court relied to correct the jury verdict. This argument is not convincing.

¶25 First, the fact that the supreme court did not discuss CrR 7.8 in *Williams-Walker* is not a persuasive distinguishing factor. There, the court held that the jury trial right included the right to be sentenced only on a basis authorized by a jury's verdict. It did so on facts that are not materially distinguishable from those here.

¶26 There, the jury verdict forms stated the "deadly weapon" enhancement, not the more serious "firearm" enhancement. Trial courts in some of the cases sentenced on the basis of the latter, not the former, enhancement. The court held that was error.

¶27 Here, the jury verdict stated Child Molestation in the Second Degree, not the more serious Child Molestation in the First Degree. The trial court sentenced on the basis of the more serious crime, not the one in the jury verdict.

¶28 The underlying principle is the same: the jury verdict authorized only a sentence based on that verdict. The court based the sentence on a crime not authorized by the jury verdict.

¶29 Second, we deal later in this opinion with the question whether CrR 7.8 is a proper remedy to correct an arguably erroneous jury verdict. That discussion deals more fully with the State's argument.

¶30 Given that there was an arguably erroneous jury verdict, we must decide whether the court had the authority to change it. Two supreme court cases provide guidance.

---

[21] *Id.*

Extracted Text

You can see that the PDF processor extracts additional information on the document such as title, page count, etc. We can use this metadata for the metadata of our chunks in the vector database.